NeuroLab Enterprise

Reporte de cumplimiento técnico frente al Documento Maestro Semana 11 y al documento Real-Time CSV ML Engine

Versión de la aplicación: 1.0.0	**Fecha:** 01/11/2026	**Estado:** Implementado
Frontend: React 19 + TypeScript + Vite	**Backend:** FastAPI + PostgreSQL	**ML:** TensorFlow / Keras

Indicador	Resultado
Reglas nucleares CSV (RN-01 a RN-05)	5 de 5 implementadas
Criterios de aceptación del MVP	7 de 7 verificables
Módulos M01 a M10	10 implementados; 6 con alcance parcial documentado
Pruebas de API	26 aprobadas
Build de frontend	TypeScript y Vite sin errores
Datos simulados en la UI	Ninguno
Almacenamiento de objetos S3	Soportado: local o bucket S3/Supabase Storage

Este reporte describe únicamente capacidades presentes en el repositorio. Los elementos no implementados se marcan como parciales o fuera del MVP; no se presentan como terminados.

1. Resumen ejecutivo

NeuroLab Enterprise es una plataforma funcional para diseñar, validar, entrenar, evaluar y versionar redes neuronales artificiales. La interfaz React consume una API FastAPI versionada en /api/v1, la API persiste el estado en PostgreSQL y un worker independiente ejecuta entrenamiento real con TensorFlow/Keras. La cola de trabajos usa Redis en producción y un backend de archivos para desarrollo; en ambos casos el proceso de ML está separado de la API.

El flujo principal se verificó de extremo a extremo: creación de proyecto, carga y validación de CSV, validación dimensional de una red Feedforward, encolado, entrenamiento Keras, telemetría por época, cancelación cooperativa, evaluación final, creación de una versión inmutable con SHA-256 y consulta posterior del resultado. La suite de API cubre autenticación, RBAC, proyectos, datasets, entrenamiento, ciclo de vida de modelos y el almacenamiento local/S3 con normalización de URLs de Supabase.

El despliegue cloud está preparado para Supabase Postgres y Storage, Vercel para el frontend y Railway para API, worker y Redis. La API y el worker usan las mismas claves lógicas de almacenamiento, por lo que el sistema funciona tanto con un volumen compartido local como con un bucket S3 privado.

Alcance honesto: el entrenamiento real está habilitado para FEEDFORWARD sobre CSV tabular. RNN, CNN y GAN se diseñan, validan y persisten, pero el worker rechaza su entrenamiento con un error 422 y una razón explícita, tal como permite el criterio de finalización del MVP.

2. Arquitectura implementada

Capa	Tecnología real	Responsabilidad
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Shell, módulos M01-M10, formularios, gráficas, descarga autenticada y reconexión SSE/WS.
API	FastAPI, Pydantic v2, SQLAlchemy 2 async, Alembic	Autenticación JWT, RBAC, validación, persistencia, encolado, streaming y auditoría.
Base de datos	PostgreSQL 16; SQLite async en desarrollo y pruebas	Usuarios, proyectos, arquitecturas, datasets, jobs, métricas, modelos y auditoría.
Cola	Redis 7 (por defecto) o filesystem	Transporte de jobs y Pub/Sub de eventos. PostgreSQL sigue siendo la fuente de verdad.
Worker ML	TensorFlow 2.x, Keras 3, scikit-learn, pandas	Preprocesamiento, construcción, entrenamiento, evaluación, artefactos y telemetría.
Almacenamiento	Volumen local o bucket S3 (Supabase Storage, MinIO, AWS S3, R2)	CSV originales y artefactos .keras, scaler.pkl, preprocessor.pkl y manifiesto JSON bajo claves datasets/ y models/.
Tiempo real	SSE y WebSocket	Snapshot inicial, epoch_end, metrics, state y error con recuperación por sondeo a la BD.
Infraestructura	Docker Compose/Nginx local; Vercel + Railway + Supabase en cloud	Proxy /api/ con SSE sin buffering en local; en cloud, frontend estático y API/worker separados.

TensorFlow nunca se importa en FastAPI. La API solo valida, persiste y encola; el worker consume y ejecuta.

3. Cumplimiento del Documento Maestro (Semana 11)

Se evalúan los requisitos funcionales, técnicos y no funcionales. La columna Estado distingue lo implementado por completo de lo que tiene alcance parcial o queda fuera del MVP.

Requisito	Implementación verificable	Estado
Interfaz React + TypeScript responsive	apps/web con React 19, TypeScript estricto, Vite y Tailwind; build de producción correcto.	Cumple
API Python versionada /api/v1	FastAPI con routers auth, projects, architectures, datasets, training-jobs, models, audit, admin y fundamentals.	Cumple
PostgreSQL para metadatos y resultados	SQLAlchemy async, esquema ORM y migración Alembic inicial; SQLite para pruebas.	Cumple
Workers aislados para TensorFlow/Keras	workers/training como proceso separado; la API no importa TensorFlow.	Cumple
Cola de trabajos	Redis con RPUSH/BLPOP y Pub/Sub; backend filesystem para desarrollo.	Cumple
Estado y métricas sin recargar	SSE y WebSocket con snapshot, epoch_end, metrics y state; la UI actualiza en vivo.	Cumple
Recuperación tras reconexión o fallo	Snapshot desde PostgreSQL, sondeo de respaldo y recuperación de jobs en processing/.	Cumple
RBAC y pertenencia a proyecto	5 roles, rol efectivo por proyecto, 404 para proyectos ajenos y dependencias de autorización por endpoint.	Cumple
Auditoría de acciones sensibles	audit_events append-only para login, proyectos, datasets, jobs, modelos y usuarios.	Cumple
Máquina de estados explícita	QUEUED, PREPARING, RUNNING, COMPLETED, FAILED y CANCELLED persistidos y validados por el worker.	Cumple
Cancelación cooperativa	Solicitud persistida + Redis/flag; Keras detiene al final de época y conserva métricas.	Cumple
Modelos inmutables y versionados	model_versions con SHA-256, artefacto por job y transiciones DRAFT/REVIEWED/APPROVED/RETIRED.	Cumple
Trazabilidad de cada ejecución	Hiperparámetros, semilla, target, checksum y versión del dataset, versión de código, métricas y auditoría.	Cumple
Validación de CSV	Extensión, tamaño, encabezados, filas, columnas, target, nulos, tipos y detección de tarea.	Cumple
Validación de entradas y errores controlados	Pydantic, rangos, enums, validación dimensional y mensajes de error legibles.	Cumple
Pruebas unitarias, integración y E2E	22 pruebas de API aprobadas y flujo E2E real con worker documentado.	Cumple
Documentación de arquitectura, API y operación	README más documentos de API, arquitectura, modelo de datos y despliegue.	Cumple
Docker y variables de entorno	Compose con Postgres, Redis, API, worker y web; Dockerfiles, entrypoint con migraciones y .env.example local/cloud.	Parcial
Despliegue cloud gestionado	Supabase Postgres + Storage, Vercel y Railway con Dockerfiles y guía operativa; configuración lista, despliegue efectivo pendiente de credenciales.	Parcial
Módulos M01-M10	Los 10 módulos existen; algunas funciones secundarias tienen alcance parcial.	Parcial
Búsqueda, filtros y etiquetas de proyecto	Listado, creación, edición, miembros y estado; búsqueda avanzada, filtros y etiquetas no están implementados.	Parcial
Prioridad de cola por permisos	Los permisos controlan quién encola; no existe prioridad de cola.	Parcial

Requisito	Implementación verificable	Estado
Reintento de entrenamientos	Reproceso manual con worker --job-id; no hay endpoint de reintento.	**Parcial**
Límites de CPU, memoria, GPU, duración y concurrencia	Límites de archivo/filas/columnas/features y timeout por configuración; no hay cuotas por usuario ni control de GPU.	**Parcial**
Exportación CSV/PDF	CSV y JSON reales, más impresión del navegador a PDF; no hay generación PDF en servidor.	**Parcial**
CI/CD, métricas Prometheus y monitoreo avanzado	Healthcheck y panel de sistema; no hay pipeline CI ni endpoint /metrics.	**Parcial**
Cifrado en tránsito y backups probados	Documentado y preparado para TLS en el proxy; los backups son procedimiento operativo, no automatización incluida.	**Parcial**
Proveedor externo de autenticación	JWT local con bcrypt; no hay IdP externo (OIDC/SAML).	**Fuera del MVP**
Ejecución de modelos o inferencia en línea	El alcance termina en registro, aprobación y descarga del artefacto.	**Fuera del MVP**

4. Cumplimiento del documento Real-Time CSV ML Engine

Las reglas de negocio RN-01 a RN-05 son el núcleo del documento. Todas están implementadas y se validan en la práctica mediante pruebas y el flujo E2E.

Regla / requisito	Implementación	Estado
RN-01: dataset verificado, encabezados explícitos, límite de tamaño y verificación de esquema	inspect_csv valida encabezados, duplicados, 10-500.000 filas, 2-500 columnas, tamaño máximo, target y tipos; guarda metadatos y checksum.	**Cumple**
RN-02: máquina de estados sin saltos	El worker escribe PREPARING antes de RUNNING y solo crea modelo al completar; estados terminales son definitivos.	**Cumple**
RN-03: aislamiento del proceso ML	Worker independiente; API solo encola. Preprocesamiento y Keras viven en workers/training.	**Cumple**
RN-04: artefactos inmutables y versionado	model.keras, scaler.pkl, preprocessor.pkl y training_manifest.json; SHA-256 en model_versions; versiones v1.0, v1.1.	**Cumple**
RN-05: cancelación cooperativa y estado parcial	Callback de Keras comprueba cancelación al inicio y fin de época; persiste métricas y marca CANCELLED sin corromper la BD.	**Cumple**
Ingesta de CSV y detección de target	El usuario selecciona el target; la API valida su existencia y detecta la tarea. No hay autodetección heurística del target.	**Parcial**
Preprocesamiento con imputación y escalado	Mediana/moda, StandardScaler, one-hot hasta 30 categorías, ordinal para el resto y split 70/15/15.	**Cumple**
Construcción dinámica del modelo	KerasModelBuilder construye capas Dense, Dropout y BatchNormalization; la salida se adapta a binaria, multiclase o regresión.	**Cumple**
Entrenamiento con callbacks y métricas por época	TelemetryCallback persiste loss/accuracy/val_loss/val_accuracy y publica epoch_end.	**Cumple**
Streaming SSE y WebSocket	Ambos canales autenticados con snapshot inicial, eventos de época, estado y error.	**Cumple**
Evaluación final	ModelEvaluator calcula pérdida, accuracy, precision, recall, F1 y matriz de confusión; para regresión MSE, RMSE, MAE y R2.	**Cumple**
Serialización .keras y preprocesador .pkl	ArtifactStorageManager guarda modelo, scaler y preprocesador, y calcula checksums individuales.	**Cumple**
Persistencia de metadatos del entrenamiento	training_manifest.json y metrics_summary incluyen dataset, semilla, hiperparámetros, split y versión de código.	**Cumple**
Descarga autorizada del modelo	GET /models/{id}/download valida pertenencia al proyecto y resuelve la clave models/... en local o en el bucket S3.	**Cumple**

5. Criterios de aceptación del MVP

Paso del criterio	Evidencia en el sistema	Estado
Usuario autenticado crea un proyecto	POST /projects con JWT y roles de escritura; UI de proyectos.	Cumple
Configura una red Feedforward válida	POST /architectures/validate y POST /architectures con validación dimensional en servidor.	Cumple
Selecciona un dataset de prueba	Carga CSV, validación de esquema, preview y selección de target.	Cumple
Inicia un entrenamiento real en TensorFlow/Keras	POST /training-jobs encola; worker ejecuta model.fit real.	Cumple
Visualiza estado y métricas sin recargar	SSE/WS con snapshot y telemetría por época; la UI actualiza en vivo.	Cumple
Consulta el resultado posteriormente	Job, métricas y modelo persisten y se consultan tras cerrar sesión.	Cumple
Conserva la versión del modelo con trazabilidad	model_versions inmutable con SHA-256, manifiesto, dataset checksum y auditoría.	Cumple

6. Estado de los módulos M01-M10

Módulo	Implementación real	Estado
M01 Dashboard	KPIs de proyectos, jobs recientes, modelos, auditoría y estado del sistema para ADMIN.	Cumple
M02 Proyectos	CRUD, miembros, roles, estado y conteos reales. Sin búsqueda avanzada, filtros ni etiquetas.	Parcial
M03 Fundamentos	Conceptos, activaciones y familias servidos por /fundamentals con contenido curricular.	Cumple
M04 Diseñador	Topologías FNN/RNN/CNN/GAN, validador dimensional, resumen de parámetros y guardado versionado.	Cumple
M05 Datasets	Carga, validación, metadatos, preview, checksum y borrado seguro con control de trazabilidad.	Cumple
M06 Entrenamientos	Configuración, cola, monitor en vivo, historial y cancelación. Sin prioridad ni reintento por API.	Parcial
M07 Evaluación	Curvas reales por época, comparación de jobs y métricas finales persistidas.	Cumple
M08 Registro de modelos	Versiones inmutables, checksum, descarga y ciclo de vida con aprobación por rol.	Cumple
M09 Reportes	Fichas y exportación CSV/JSON con datos persistidos; PDF vía impresión del navegador.	Parcial
M10 Administración	Usuarios, roles, estados, estado de BD/Redis/cola y auditoría. Sin cuotas de CPU/GPU.	Parcial

7. Validación, seguridad y operación

Área	Implementación
Autenticación	bcrypt + JWT HS256; recarga del usuario en cada request; cuentas INACTIVE/SUSPENDED sin acceso.
Autorización	Roles ADMIN, PROJECT_LEAD, AI_ENGINEER, REVIEWER y VIEWER; permisos de escritura y aprobación separados.
Aislamiento horizontal	Sin membresía se devuelve 404 para no revelar la existencia de proyectos ajenos.
Rate limit	Login por IP y minuto; en memoria por proceso, no distribuido.
Validación	Pydantic, enums, rangos, validación dimensional y validación de CSV.
Rutas de archivos	Claves datasets/ y models/ validadas contra path traversal y resueltas en DATA_DIR/MODEL_DIR o en el bucket S3.
CORS	Lista explícita de orígenes y regex opcional para dominios de preview de Vercel.
Auditoría	Eventos append-only con actor, proyecto, acción, metadatos y fecha.
Secretos	Variables de entorno del servidor; el bundle React solo recibe la URL de API.
Cabeceras	X-Content-Type-Options, X-Frame-Options y Referrer-Policy.
Migraciones	Alembic como fuente del esquema; create_all solo en desarrollo.

8. Evidencia de verificación

Verificación	Resultado
python -m pytest -q en services/api	26 pruebas aprobadas (incluye almacenamiento local y normalización de URL Supabase)
npm run build en apps/web	tsc y vite build correctos
alembic check	Sin operaciones pendientes sobre el esquema actual
E2E API - cola filesystem - worker TF/Keras	Job COMPLETED, métricas persistidas y modelo v1.0 DRAFT con SHA-256
Cancelación E2E	Job CANCELLED con métricas parciales conservadas
SSE	Snapshot inicial y evento de estado terminal recibidos
Docker Compose	Definido y revisado estáticamente; runtime no validado en este entorno por falta de daemon
Redis runtime	No disponible en el entorno local; el E2E usó el backend filesystem
Supabase/Vercel/Railway	Configuración y guía completas; falta ejecutar el despliegue con credenciales reales

9. Limitaciones y trabajo futuro

- Entrenamiento real solo para FEEDFORWARD tabular; RNN/CNN/GAN quedan documentadas y validadas.

- El modo local exige que API y worker compartan volumen; para hosts separados debe usarse STORAGE_BACKEND=s3 (Supabase Storage, MinIO, S3 o R2).

- El despliegue cloud está configurado y documentado, pero no ejecutado: requiere credenciales de Supabase, Vercel y Railway.

- El plan Free Trial de Railway dura 30 días, incluye 5 USD y limita cada servicio a 1 GB de RAM; TensorFlow puede agotar la memoria con datasets grandes.

- Sin cuotas de CPU, memoria, GPU ni concurrencia por usuario/proyecto.

- Sin prioridad de cola ni endpoint de reintento; el reproceso es operativo con worker --job-id.
- Rate limit de login en memoria por proceso.
- Sin CI/CD ni endpoint de métricas Prometheus.
- Sin generación de PDF en servidor; se usa impresión del navegador.
- Sin autenticación federada (OIDC/SAML) ni refresh tokens.
- La validación de runtime con Docker/PostgreSQL/Redis queda pendiente en un entorno con daemon disponible.

10. Conclusión

La implementación cumple el criterio de finalización del MVP del Documento Maestro: un usuario autenticado crea un proyecto, configura una red Feedforward válida, selecciona un dataset, inicia un entrenamiento real con TensorFlow/Keras, observa estado y métricas sin recargar, consulta el resultado posteriormente y conserva una versión inmutable con trazabilidad. También cumple las cinco reglas de negocio del documento CSV y su flujo de ingesta, preprocesamiento, entrenamiento, evaluación y versionado.

Las brechas restantes son de endurecimiento empresarial y de alcance ampliado, no de funcionamiento del flujo principal: cuotas de recursos, prioridad/reintento de cola, observabilidad avanzada, CI/CD, autenticación federada y ejecución efectiva del despliegue cloud (la configuración ya existe). Están identificadas de forma explícita para que la siguiente fase las aborde sin presentar como terminado lo que aún no lo está.